

Compte Rendu de Projet

ALGORITHMIQUE

Bataille Navale

La bataille navale										
	1	2	3	4	5	6	7	8	9	10
A										
B										
C										
D										
E										
F										
G										
H										
I										
J										

1 porte avion

 5 cases

1 croiseur

 4 cases

2 sous-marins
 
 3 cases + 3 cases

1 torpilleur

 2 cases

○ à l'eau
 X touché

Le joueur qui touche ou coule recommence

Sommaire :

PRESENTATION	3
THEME	3
OBJECTIFS	3
ANALYSE DU SUJET	3
REGLE DU JEU	3
METHODOLOGIE	4
APPROCHE	4
REPARTITION DU TRAVAIL	4
OUTILS ET RESSOURCES	4
REALISATION	4
REPRESENTATION DU JEU	4
EXPLICATION DES FONCTIONS	5
ANALYSE DETAILLEE DU CODE	7
GESTION DES ERREURS ET CAS LIMITES	8
DEROULEMENT D'UNE PARTIE	8
SYNTHESE	9
BILAN GENERAL	9
DIFFICULTES RENCONTREES	9
LIMITES IDENTIFIEES	10
AMELIORATIONS POSSIBLES	10
CONCLUSION	10

Présentation

Thème

Ce projet consiste à réaliser un jeu de bataille navale en langage PHP. Ce projet a été développé dans le cadre du cours d'algorithmique et vise à mettre en pratique les notions de cours suivantes :

- Manipulation de tableaux multidimensionnels (grilles de jeu 10×10)
- Structuration du code en fonctions réutilisables
- Passage de paramètres par référence en PHP
- Gestion des entrées/sorties en ligne de commande
- Implémentation d'une logique de jeu complète avec boucle de partie

Objectifs

L'objectif de ce projet est de réaliser un jeu de bataille navale classique : 2 joueurs qui s'affrontent à tour de rôle pour couler les navires adverses sur une grille cachée. L'ensemble du projet est contenu dans un seul fichier PHP (affichetab.php) exécutable en mode CLI.

L'objectif pédagogique est de décomposer un problème complexe en sous-problèmes, à les résoudre indépendamment à l'aide de fonctions, puis à les assembler en un programme cohérent et fonctionnel.

Analyse du sujet

Règle du jeu

La Bataille Navale (ou "Touché-Coulé") est un jeu de stratégie opposant deux adversaires. L'objectif est simple : localiser et détruire l'intégralité de la flotte adverse dissimulée sur une grille avant que ses propres navires ne soient coulés.

Pour le développement de cet algorithme en PHP, nous nous basons sur les règles classiques suivantes :

- **Le Plateau** : Chaque joueur dispose de deux grilles de 10 x 10 cases (indexé de 1 à 10).
- **La Flotte** : Elle est composée de navires de tailles variables (généralement de 2 à 5 cases). Une règle de gestion de collision stricte est appliquée : deux bateaux ne peuvent pas se chevaucher.
- **Le Système de Tour** : Le joueur annonce une coordonnée cible, l'adversaire vérifie l'état de la case et répond :
 - Louper : Le tir ne touche aucun navire.
 - Touché : Le tir atteint une section d'un navire encore à flot.
 - Touché-Coulé : Toutes les sections d'un navire ont été atteintes.

- **Condition de Victoire** : La partie prend fin dès qu'un joueur a totalisé un nombre de tirs réussis correspondant au nombre cumulé de cases de la flotte adverse.
-

Méthodologie

Approche

- Décomposition modulaire : Séparation du code en trois blocs : Initialisation, les fonctions de jeu, et Affichage.
- Développement itératif :
 1. Création de la grille.
 2. Placement manuel puis aléatoire.
 3. Système de tir.
 4. Boucle de fin de partie.

Répartition du travail

Se travaille étant en binôme nous nous sommes toutes deux réparties les différentes fonctions à réaliser, puis nous prenons des temps pour fusionner nos travaux ensemble et réaliser les phases de tests de notre jeu.

Outils et Ressources

- **Langage** : PHP
 - **Environnement** : VS Code / PHPStorm ou Onlinegdb.
 - **Documentation** : site officiel de PHP
-

Réalisation

Représentation du jeu

1. La grille

La grille de jeu est modélisée par un tableau PHP bidimensionnel de 10 lignes et 10 colonnes. Chaque case peut contenir l'une des valeurs suivantes :

- 0 : case vide, aucun navire
- "X" : case vide déjà tirée (tir à l'eau)
- Valeur négative (ex : -2, -3) : case de navire déjà touchée

L'utilisation de valeurs négatives pour les cases touchées est une approche ingénieuse car elle permet de conserver l'identifiant du navire (pour mettre à jour les points de vie) tout en marquant la case comme déjà atteinte.

2. Les bateaux

Conformément au sujet fourni, chaque joueur doit positionner les navires suivants sur sa grille de 10×10 cases :

Navire	Taille	Code valeur
Torpilleur	2 cases	2
Sous-marin 1	3 cases	3
Sous-marin 2	3 cases	9
Croiseur	4 cases	4
Porte-avion	5 cases	5

Chaque navire peut être placé horizontalement ou verticalement, sans chevauchement avec un autre navire. Les navires ne doivent pas dépasser les limites de la grille.

3. Gestion des vies

Chaque joueur dispose d'un tableau de vies à 5 entrées, correspondant aux 5 navires :

```
$vieJ1 = [2, 3, 3, 4, 5]; // torpilleur, sous-marin 1, sous-  
marin 2, croiseur, porte-avion
```

Lorsqu'une case est touchée, la valeur de la case est récupérée pour identifier le navire, et l'entrée correspondante dans le tableau des vies est décrémentée. La partie s'arrête lorsque la somme des vies d'un joueur atteint zéro.

4. Architecture du code

Le programme suit une architecture procédurale, découpée en fonctions spécialisées. Cette approche favorise la lisibilité et la réutilisabilité du code. La partie principale (en bas du fichier) orchestre l'ensemble : initialisation des grilles, placement des navires, puis boucle de jeu.

Explication des fonctions

Le code source est organisé autour de 8 fonctions principales. Le tableau ci-dessous présente leur rôle respectif :

Fonction	Rôle
<code>initialisegrille()</code>	Crée et retourne un tableau 2D de 10×10 cases initialisées à 0.
<code>afficheTab(\$tab, \$m, \$n)</code>	Affiche la grille dans la console ligne par ligne.
<code>positionnetorpilleur(&\$grille)</code>	Permet au joueur de placer le torpilleur (2 cases, code 2) sur la grille.
<code>positionnesous_marins1(&\$grille)</code>	Place le premier sous-marin (3 cases, code 3) sur la grille.
<code>positionnesous_marins2(&\$grille)</code>	Place le deuxième sous-marin (3 cases, code 9) sur la grille.
<code>positionnecroiseur(&\$grille)</code>	Place le croiseur (4 cases, code 4) sur la grille.
<code>positionneporte_avion(&\$grille)</code>	Place le porte-avion (5 cases, code 5) sur la grille.
<code>tire(&\$grille, &\$vie)</code>	Gère un tir : détection de touché, raté ou déjà touché, mise à jour des vies.

1. Fonctions de positionnement

Les cinq fonctions de positionnement (torpilleur, sous-marins, croiseur, porte-avion) suivent toutes le même patron :

- Lecture des coordonnées (ligne, colonne) et de l'orientation (horizontal/vertical) via `readline()`.
- Vérification que le navire ne dépasse pas les limites de la grille.
- Vérification qu'aucune case n'est déjà occupée.
- Écriture de l'identifiant du navire dans toutes les cases concernées.
- En cas d'erreur, appel récursif de la fonction pour redemander les coordonnées.

2. Fonction `tire()`

La fonction `tire()` est le cœur du jeu. Elle prend par référence la grille de l'adversaire et son tableau de vies, lit les coordonnées du tir, puis applique la logique suivante :

```

Si grille[l][c] < 0 ou == 'X' → Zone déjà touchée
Si grille[l][c] != 0          → Touché ! (valeur multipliée par -1)
Sinon                        → Loupé (case remplacée par 'X')
```

Analyse détaillée du code

1. Initialisation de la grille

La fonction `initialisegrille()` crée un tableau 2D en PHP. La déclaration explicite de la variable en tableau vide (`$grille = array()`) est obligatoire avant d'utiliser les indices, sous peine d'erreur de type.

```
function initialisegrille(){  
    $grille = array();  
    for($i=0; $i<10; $i++){  
        for($j=0; $j<10; $j++){  
            $grille[$i][$j] = 0;  
        }  
    }  
    return $grille;  
}
```

2. Passage par référence

Un point technique important est l'utilisation du passage par référence (symbole `&`) pour les paramètres `$grille` et `$vie` dans les fonctions `tire()` et `positionne*`. Sans cela, PHP passerait une copie du tableau, et les modifications ne seraient pas répercutées sur les variables originales du programme principal.

```
function tire(&$grille, &$vie){ ... }  
function positionnetorpilleur(&$grille){ ... }
```

3. Détection des tirs dupliqués

La condition de détection d'une zone déjà touchée utilise un OU logique :

```
if ($grille[$l][$c] == "X" or $grille[$l][$c] < 0) {  
    echo "Zone déjà touché";  
    return;  
}
```

Cette double vérification couvre les deux cas : une case à l'eau ("X") et une case de navire déjà touchée (valeur négative). Le `return` sans argument permet de sortir de la fonction sans incrémenter de tour.

4. Boucle de jeu principale

La boucle de jeu utilise `while` avec une condition portant sur `array_sum()` des tableaux de vies. Dès que la somme des vies d'un joueur atteint 0, la partie s'arrête :

```
while ((array_sum($vieJ1) > 0) && (array_sum($vieJ2) > 0)) {  
    // J1 tire sur grille J2  
    // J2 tire sur grille J1  
}
```

Une vérification intermédiaire après le tir du Joueur 1 permet d'éviter que le Joueur 2 tire inutilement si la partie est déjà terminée.

Gestion des erreurs et cas limites

1. Dépassement de la grille

Chaque fonction de positionnement vérifie que le navire ne dépasse pas les bornes de la grille avant d'écrire. Par exemple pour le torpilleur en horizontal :

```
if ($c + 1 <= 9) { // La colonne de fin ne dépasse pas l'index 9  
    // placement autorisé  
} else {  
    echo "Impossible : le torpilleur dépasse de la grille."  
    positionnetorpilleur($grille); //appel récursif  
}
```

2. Chevauchement de navires

Avant de placer un navire, la fonction vérifie que toutes les cases concernées contiennent bien la valeur 0 (case vide). Si une case est déjà occupée, le joueur est invité à recommencer via un appel récursif.

3. Zone déjà ciblée

La fonction tire() retourne immédiatement si la case visée a déjà été ciblée, sans consommer le tour. Cela évite de pénaliser le joueur pour une erreur de saisie, bien que l'on puisse argumenter qu'une consommation de tour serait plus fidèle aux règles originales.

Déroulement d'une partie

1. Phase d'initialisation

Au démarrage, le programme crée deux grilles vierges (\$grilleJ1 et \$grilleJ2) et deux tableaux de vies initialisés à [2, 3, 3, 4, 5]. Puis il demande successivement aux deux joueurs de placer leurs 5 navires.

Pour chaque navire, le joueur saisit :

- Le numéro de ligne (0 à 9)
- Le numéro de colonne (0 à 9)
- L'orientation : horizontal (o) ou vertical (n)

Après chaque placement, la grille est affichée pour que le joueur visualise l'état de sa flotte.

2. Phase de jeu

La boucle de jeu alterne les tours des deux joueurs. À chaque tour, le joueur actif saisit les coordonnées de son tir. Le résultat est affiché immédiatement, ainsi que l'état des vies restantes pour les cinq navires adverses.

Exemple d'affichage après un tir :

```
Touché !  
Vies restantes : Torpilleur = 1, Sous-marin 1 = 3, Sous-marin 2  
= 3, Croiseur = 4, Porte-avion = 5
```

3. Fin de partie

La partie prend fin dès que la somme des vies d'un joueur atteint 0. Un message de victoire est affiché et la boucle est interrompue avec break.

Exemple :

```
Joueur 2 est éliminé ! Joueur 1 a gagné !
```

Synthèse

Bilan général

Le projet remplit son objectif principal : proposer une implémentation fonctionnelle et jouable de la Bataille Navale en PHP CLI. Les règles de base sont respectées, la logique de jeu est opérationnelle, et le code est structuré de manière claire grâce au découpage en fonctions.

Les notions algorithmiques vues en cours sont bien utilisées : tableaux bidimensionnels, fonctions avec passage par référence, structures conditionnelles imbriquées, boucles, et récursivité.

Difficultés rencontrées

Plusieurs difficultés techniques ont été rencontrées lors du développement :

- La fusion des différentes fonctions entre elles
- La gestion des erreurs
- Essayer de couvrir tous les cas possible (erreurs, chevauchement de bateaux, ect...)

Limites identifiées

Le projet dans sa forme actuelle présente quelques limites à prendre en considération :

- La récursivité pour la gestion des erreurs de placement peut entraîner un débordement de pile si le joueur fait de très nombreuses erreurs successives. Une boucle while serait plus robuste.
- Il n'y a pas de validation de la plage des coordonnées saisies (ex: entrée négative ou supérieure à 9), ce qui peut provoquer un comportement inattendu.
- Les deux grilles sont visibles dans la même session, ce qui n'est pas idéal pour un jeu en face-à-face sur une même machine.

Améliorations possibles

Plusieurs axes d'amélioration pourraient enrichir ce projet dans une version future :

- Validation des entrées : vérifier que les coordonnées saisies sont bien des entiers compris entre 0 et 9 avant de les utiliser, pour éviter tout comportement imprévu.
- Remplacement de la récursivité par des boucles while pour la gestion des erreurs de placement, ce qui éliminerait tout risque de débordement de pile.
- Ajout du bonus « rejouer en cas de touché » : selon les règles originales, un joueur qui touche un navire a le droit de tirer à nouveau.
- Grille masquée pour l'adversaire : afficher uniquement les tirs effectués (X et cases négatives transformées en symboles neutres) sans révéler la position des navires restants.
- Mode de placement automatique (aléatoire) pour accélérer la phase de mise en place.
- Interface Web ou graphique pour une expérience de jeu plus agréable qu'en ligne de commande.
- Ajout d'un mode solo avec un adversaire contrôlé par l'ordinateur (IA aléatoire ou ciblée).

Conclusion

Ce mini-projet constitue une application concrète et complète des concepts vus en cours d'algorithmique. Il démontre qu'il est possible de développer un jeu interactif entier avec des outils fondamentaux : tableaux, fonctions, boucles et conditions.

Le langage PHP, bien que moins courant pour ce type d'application CLI, s'est révélé parfaitement adapté grâce à sa gestion native des tableaux associatifs et à la fonction `readline()` pour les entrées utilisateur.

La démarche de développement adoptée – analyse du sujet, conception du modèle de données, découpage en fonctions, implémentation et tests – c'est montré satisfaisante et complète.